

CSV Data

CSCI 1101

2024-03-08

Tables

Tabular data is very common:

name	diet	weight
Whiskers	rat chow	300.0
Mr. Squeaky	swiss cheese	450.0
Pinky	rat chow	320.0
Fluffball	swiss cheese	500.0

In python we can represent a **table** using a list of tuples:

```
# data : list (tuple [str, str, float])
data = \
[   #   name           ,   diet           ,   weight
    ('Whiskers'        , 'rat chow'      , 300.0) ,
    ('Mr. Squeaky'     , 'swiss cheese'  , 450.0) ,
    ('Pinky'           , 'rat chow'      , 320.0) ,
    ('Fluffball'       , 'swiss cheese'  , 500.0)
]
```

Working With Tabular Data

Many programs work with tabular data in the form of **spreadsheets**.

They generally store their data in a file, and have a workflow that involves:

loading data from a file,

editing the data somehow,

storing the data back to a file.

Each spreadsheet program has its own native file format, but nearly all support the very simple format of **comma-separated values** (CSV).

CSV Data

The basic idea of the CSV format is that:

- each *row* of a table is stored as a *line* of text,
- the *cells* of a row are separated by commas,
- the first row of the table is a *header*, containing the *column names*.

Here is our rat diet table represented in CSV format, stored in a file `diets.csv`:

```
name,diet,weight
Whiskers,rat chow,300.0
Mr. Squeaky,swiss cheese,450.0
Pinky,rat chow,320.0
Fluffball,swiss cheese,500.0
```

Python CSV Library

Python includes a library for reading and writing CSV files.

To use it we must first import it:

```
import csv
```

Then we can read and write CSV files similarly to the way we do plain text files.

Reading CSV Data

To read in CSV data use a `csv.reader` object, which is an iterator that breaks each line of a CSV file into a list of cells:

```
def load_csv(path) :  
    ''' sig: (str) --> list (list str) '''  
    data = []                # accumulator to store file contents  
    with open(path , 'r') as file : # open a file for reading  
        reader = csv.reader(file) # create a csv reader  
        for row in reader : # iterate over CSV reader  
            data.append(row)  
    return data
```

Decoding CSV Data

Loading a CSV file will produce a list of lists of strings:

```
csv_data = load_csv('diets.csv')
```

```
diets_header = csv_data[0]
```

We expect this data to form a table,

but we need to make sure that:

- all rows has the same number of cells,
- except for the first row, which consists of string column headings, the corresponding cell in each row represents data of the same *type*.

For example, our rat diet table should have three columns, respectively of type `str`, `str`, and `float`.

Parsing a Table Row

If we know the number and type of table columns then we can write a **row parser** to turn a list of strings into a tuple of the right size and shape:

```
def parse_row(parsers , cells) :  
    ''' sig: (list fun , list str) --> tuple '''  
    results = () # the empty tuple with type tuple []  
    for (parser, cell) in zip(parsers, cells, strict=True) :  
        parsed_cell = parser(cell) # parse the cell  
        # concatenate a *singleton tuple* to the results:  
        results += (parsed_cell , )  
    return results
```

For example:

```
> parse_row([str, str, float] , csv_data[1])  
('Whiskers', 'rat chow', 300.0)
```


Parsing a Table

To parse a table we *map* a row parser over the rows of the table:

```
def parse_table(parsers , rows) :  
    ''' (list fun , list (list str)) --> list tuple '''  
    # the `map` pattern:  
    acc = []  
    for cells in rows :  
        acc.append(parse_row(parsers , cells))  
    return acc
```

This lets us reconstitute a table from the CSV data:

```
diets = parse_table([str,str,float] , csv_data[1:])
```

Editing a Table

Let's add a new rat to our study:

```
# append a new tuple [str, str, float]
# to our list (tuple [str, str, float]):
diets.append(('Ricky' , 'rat chow' , 280.0))
```

Writing CSV Data

To write CSV data, use a `csv.writer` object, which has the following methods:

`writerow` takes an iterator of any data that can be stringified,

`writerows` takes an iterator of any data that can be `writerowed`.

```
def store_csv(path , headers , data) :  
    with open(path , 'w' , newline='') as file :  
        writer = csv.writer(file) # create a csv writer  
        writer.writerow(headers)  # write the header row  
        writer.writerows(data)    # write the table rows
```

For example:

```
> store_csv('new_diets.csv' , diets_header , diets)
```

To Do

Finish on *Project 2 - What Does the Fox Say?* on CodeRunner by midnight tonight.

Finish *Homework 7: While loops and Files* on CodeRunner by 11:00AM Thursday, March 28.

Have a safe and happy spring break.